

Prompt to Elephant

1. Take this design document as well as the feedback and give me how you would update the design. Give a summary of everything you give me. (Plus attached HW1 and feedback)

Elephant Summary

1. Your revised project should be a minimal extension of the tutorial architecture, with Guess the City as the primary required game and Guess the Landmark as a second mode demonstrating code reuse. ContentView, ConversationView, and ChatView remain responsible for presentation; AppViewModel owns prompts, selected game, conversation state, errors, and game lifecycle; Rein continues to own networking and SSE processing; and /llmchat remains the primary backend interface. This preserves the architecture your original document explicitly intended to maintain. When a game begins, AppViewModel sends an application-controlled initialization system message defining the selected game's rules. That initialization prompt is distinct from the user-editable system prompt inherited from the tutorial. Guess the City instructs the LLM to secretly choose one city, provide clues, continue after incorrect guesses, and reveal the city's latitude and longitude after a correct guess. Guess the Landmark uses the same infrastructure but substitutes a landmark-specific initialization prompt. For ordinary sends, AppViewModel constructs one [Message] array containing a system-role message when systemPrompt is nonempty and a user-role message when userPrompt is nonempty. That entire array is passed to Reinonce, producing exactly one /llmchat request. larmijo The assistant response continues to use the tutorial's SSE mechanism: an assistant Chat is created and its observable content is updated incrementally as tokens arrive. Switching games clears the previous conversation/context before changing selectedGame and initializing the new game, preventing City context from leaking into Landmark and vice versa. Clear performs only the clearing operation without changing the selected game. The Send button is disabled when both input fields are empty or while a response is actively streaming, and the existing tutorial error mechanism should handle networking and backend failures. So the key change from your original design is not a big architectural rewrite. It's making the design more precise, more tutorial-aligned, and less ambiguous: one architecture, one networking path, one streaming mechanism, one AppViewModel controlling the lifecycle, and different games defined primarily by their initialization prompts. That should also give you a much cleaner blueprint for building the project from the two tutorials.

Prompt to Goldfish

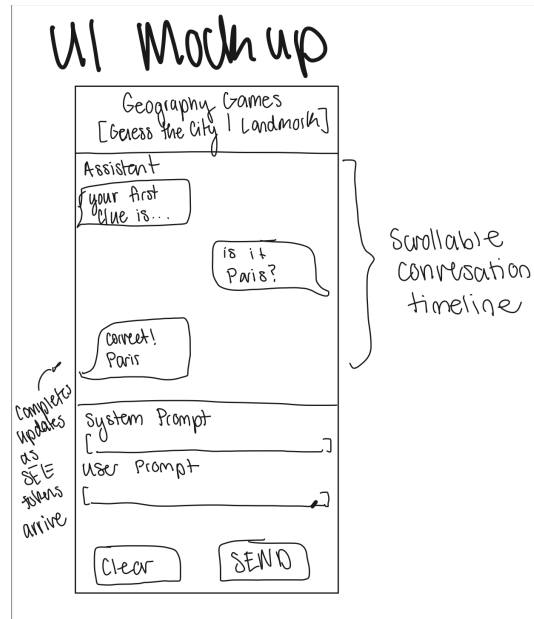
1. Based on this summary {insert previous summer} critique this design and change/add anything you think is needed. Give a summary of all changes once you are done.

Goldfish Summary

1. Your fundamental design does not need to be rewritten. I would keep the tutorial's ContentView → AppViewModel → Rein → backend architecture and its existing SSE conversation mechanism. The main changes I would make are to remove ambiguity: explicitly define startGame(), send(), clear(), and switchGame(); distinguish the application-controlled initialization prompt from the editable tutorial system prompt; represent City/Landmark through a small Game type; introduce explicit isStreaming state; define startup behavior; clear both frontend and backend context when switching; prevent switching/sending during an active stream; trim empty prompts; ensure streaming state resets after errors; and keep all game-specific behavior out of Rein and the shared conversation views. The resulting principle is: One UI architecture, one ViewModel, one networking layer, one /llmchat path, one SSE mechanism, and one conversation representation. The games differ primarily through their initialization prompts. That is the direction I would use for the final design document because it is both easier to implement from your tutorials and much clearer about exactly where every responsibility belongs.

Updated Final Design

1. App Overview
 - a. My app will contain two games: Guess the City and Guess the Landmark. In Guess the City, the LLM chooses a mystery city without revealing its name and provides clues until the user guesses correctly. After an incorrect guess, the LLM will continue providing clues without revealing the answer. Once the correct city is guessed, the LLM will congratulate the user, reveal the city's name, provide its latitude and longitude, and ask whether the user would like to play again. In Guess the Landmark, the LLM will similarly choose a famous landmark without revealing its name and provide clues until the user identifies it. Once the landmark is guessed correctly, the LLM will reveal the landmark's name and location and ask whether the user would like to play again. Both games will use the same UI, ViewModel, networking layer, backend infrastructure, conversation representation, /llmchat endpoint, and SSE streaming mechanism. The primary difference between the two games will be the application-controlled initialization system prompt sent when the game begins. The user will select which game to play using a game selector at the top of the screen. The application will build directly on the architecture, separation of concerns, naming, and code structure established in the llmPrompt and llmChat tutorials. I will extend the tutorial code rather than replacing its architecture or changing the existing /llmchat API or OpenAIRequest format. This preserves the same architectural direction established in my original design.



2. Front-End Design

a. ContentView

- i. The front end will continue to build directly on the structure established by the `llmPrompt` and `llmChat` tutorials. `ContentView` will contain:
 1. A game selector for Guess the City and Guess the Landmark
 2. A system prompt input
 3. A user prompt input
 4. A Send button
 5. A Clear button
 6. The existing `ConversationView`
- ii. The game selector determines which game the application is currently playing but does not introduce separate views or networking implementations for each game. `ContentView` will primarily be responsible for displaying state and forwarding user actions to `AppViewModel`. Game rules, networking, SSE processing, and conversation management will not be implemented directly inside `ContentView`.

b. ConversationView and ChatView

- i. `ConversationView` will continue to display the conversation as a timeline of `ChatView` objects. The timeline will display user messages and LLM completions. The existing observable `Chat.content` design from the tutorial will continue to support streamed responses. As SSE tokens arrive, the assistant's `Chat.content` will change and the existing `ChatView` will update automatically. No City- or Landmark-specific behavior will be placed inside `ConversationView` or `ChatView`. Therefore: `ConversationView` →

ChatView, only displays conversation state. It does not need to know whether the current conversation is Guess the City or Guess the Landmark. This preserves the original design's use of observable Chat.content for streamed SSE completions.

c. Game Type

- i. A small Game type will represent the selected game. Conceptually:
 1. Game
 2. |— city
 3. |— landmark
- ii. Game will allow AppViewModel to determine which initialization prompt should be used without introducing separate ViewModels or networking layers for each game. The game type should remain intentionally small. It represents which game is selected, while AppViewModel controls the actual game lifecycle.

d. AppViewModel

- i. AppViewModel will remain responsible for application state and coordinating application behavior. It will store:
 1. AppViewModel
 2. |— rein: Rein
 3. |— systemPrompt: String
 4. |— userPrompt: String
 5. |— selectedGame: Game
 6. |— conversation: [Chat]
 7. |— isStreaming: Bool
 8. |— errMsg: String
 9. |— showError: Bool
- ii. AppViewModel will:
 1. Maintain the currently selected game.
 2. Maintain the editable system and user prompt fields.
 3. Maintain the displayed conversation.
 4. Track whether an SSE response is currently streaming.
 5. Initialize games.
 6. Coordinate Send operations.
 7. Coordinate Clear operations.
 8. Coordinate game switching.
 9. Handle errors returned by networking operations.
- iii. The four main application operations will be explicitly defined as:
 1. startGame()
 2. send()
 3. clear()

4. switchGame()

3. Initialization Prompts

a. Application-Controlled Initialization Prompt

- i. Each game will have an application-controlled initialization system prompt. This prompt is different from the editable systemPrompt field displayed in the tutorial UI. The application-controlled prompt establishes the rules of the game and is generated internally based on selectedGame. The user does not directly edit this prompt. Therefore there are two distinct concepts: Application-controlled initialization prompt and User-editable systemPrompt. The initialization prompt establishes game behavior. The editable systemPrompt allows the user to send an additional system-role message through the tutorial interface.

b. Guess the City Initialization

- i. When selectedGame == .city, startGame() will construct a system-role initialization message instructing the LLM to:
 1. Choose exactly one mystery city.
 2. Do not initially reveal the city's name.
 3. Provide the user with an initial clue.
 4. Treat subsequent user messages as guesses or questions.
 5. Respond to incorrect guesses without revealing the answer.
 6. Continue providing useful clues.
 7. Congratulate the user after the correct city is guessed.
 8. Reveal the city's name after the correct guess.
 9. Provide the city's latitude and longitude.
 10. Invite the user to play again.
- ii. The initialization message will be sent through the same /llmchat infrastructure used for normal messages. The backend's existing conversation context will preserve the previous turns so that subsequent user guesses remain part of the same game. These requirements refine the City behavior already defined in the original design.

c. Guess the Landmark Initialization

- i. Guess the Landmark will reuse all infrastructure used by Guess the City. The primary difference will be its initialization system message. When: selectedGame == .landmark startGame() will create a system-role message instructing the LLM to select a famous landmark without initially revealing its name and provide clues based on its appearance, history, location, or significance. After the user correctly identifies the landmark, the LLM will reveal its name and location and invite the user to play another round. This preserves the original design principle that City and Landmark reuse the same infrastructure.

4. startGame()

- a. startGame() will be responsible for beginning the currently selected game. Conceptually: startGame() → inspect selectedGame → construct appropriate initialization Message → set isStreaming = true → create assistant Chat → Rein sends /llmchat request → process SSE → assistant Chat.content update → set isStreaming = false. startGame() will not implement networking itself. It will use Rein for the actual request. Likewise, Rein will not determine which game is being played. AppViewModel selects the appropriate initialization message before calling Rein. This keeps game-specific behavior out of the networking layer.

5. send()

a. Building Messages

- i. The system and user input fields will remain independent. The user may enter:
 - 1. Only a system prompt
 - 2. Only a user prompt
 - 3. Both prompts
- ii. When Send is pressed, send() will first trim leading and trailing whitespace from both fields. For example, an input containing only: " " will be treated as empty. send() will then construct one [Message] array. If the trimmed system prompt is nonempty, a system-role Message will be appended. If the trimmed user prompt is nonempty, a user-role Message will be appended. Therefore: systemPrompt → if nonempty → Message(role: system) → userPrompt → if nonempty → Message(role: user) → [Message] → Rein.llmPrompt(...) → ONE POST /llmchat. This clarifies the wording of the original design while preserving its requirement that both inputs result in one request rather than separate requests.

b. Send Conditions

- i. send() will not begin another request when: isStreaming == true. The Send button will therefore be disabled when: both trimmed inputs are empty OR isStreaming == true. When the request begins: isStreaming = true and the Send button will display the tutorial's progress indicator. After the SSE response finishes: isStreaming = false. The input fields will be cleared after their contents have been captured for the request. This preserves the original design's intention to prevent overlapping Send operations.

6. SSE Streaming

- a. The existing SSE implementation from the tutorial will continue to be used. Before requesting a completion, an assistant Chat object will be added to the conversation. Rein will then make the /llmchat request and process the SSE

response. As completion tokens arrive: SSE token → Chat.content → observable state changes → ChatView updates. The application will therefore display the LLM response incrementally instead of waiting for the entire completion. No separate streaming implementation will be created for either game. Both City and Landmark use the same SSE mechanism. This preserves the streaming architecture described in the original design.

7. isStreaming State

- a. An explicit: isStreaming: Bool, will represent whether the application currently has an active streamed completion. Before an LLM request begins: isStreaming = true. After the stream successfully finishes: isStreaming = false. If the request or SSE stream fails: isStreaming = false, must still occur. This prevents the application from becoming permanently stuck in a loading state after an error.

While isStreaming is true:

- i. Send will be disabled.
- ii. The game selector will be disabled.
- iii. Another game cannot be initialized.
- iv. Another LLM request cannot begin.

8. clear()

- a. clear() will clear the current conversation context while leaving the selected game unchanged. The operation will coordinate both backend and frontend state. Conceptually: clear() → clear backend conversation context → backend succeeds → conversation.removeAll(). The frontend conversation will therefore remain synchronized with the backend conversation context. clear() will not change: selectedGame. For example, if the user is playing Guess the City and presses Clear, the application remains configured for Guess the City. clear() cannot begin while isStreaming == true.

9. switchGame()

- a. switchGame() will explicitly handle changing from City to Landmark or Landmark to City. Conversation context from one game must not affect the other. The game selector will therefore be disabled while: isStreaming == true. When the user selects another game and no stream is active, switchGame() will: User selects another game → clear backend conversation context → clear frontend conversation → change selectedGame → startGame() → send new initialization prompt → new game's first clue streams into UI → new game's first clue streams into UI. The previous game's context will therefore be removed before the new game's initialization prompt is sent. This formalizes the context isolation intended by the game-switching portion of the original design.

10. Error Handling

- a. AppViewModel will continue to maintain:
 - i. errMsg: String

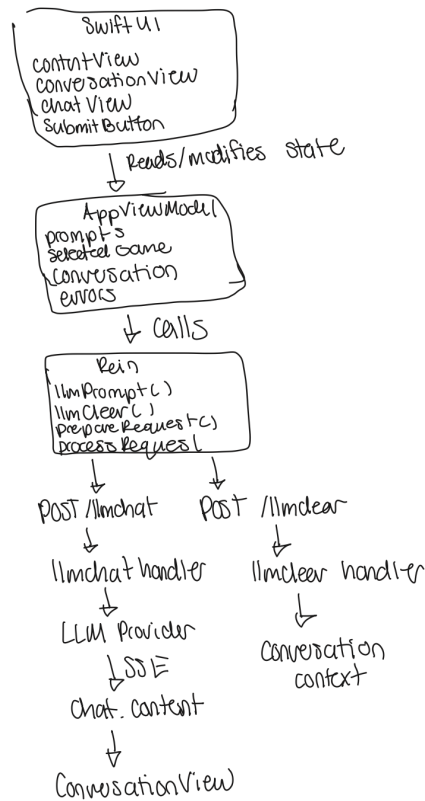
- ii. showError: Bool
- b. If game initialization, sending, clearing, networking, or SSE processing fails, the ViewModel will store an appropriate error message and cause the SwiftUI interface to present the error using the tutorial's existing error-display approach. Most importantly, an error during a streamed operation must reset: isStreaming = false, so that the user can continue interacting with the application. The error path will therefore behave conceptually as: Request begins → isStreaming = true → ERROR → errMsg updated, showError = true, isStreaming = false

11. Rein

- a. Rein will remain the networking layer established by the tutorials. Its responsibilities will include:
 - i. Creating/sending HTTP requests
 - ii. Communicating with /llmchat
 - iii. Processing SSE responses
 - iv. Updating streamed completion content
 - v. Performing any required backend conversation-clearing request
- b. Rein will not contain game-specific behavior. It will not:
 - i. choose City vs Landmark
 - ii. construct game rules
 - iii. decide when a game begins
 - iv. decide when the user won
 - v. control selectedGame
 - vi. control SwiftUI
- c. Those responsibilities belong either to AppViewModel or, for the conversational game behavior itself, to the LLM.

12. Frontend and Backend Code Structure

- a. The most important ownership rule will be: SwiftUI renders state, AppViewModel controls the application and game lifecycle, Rein handles networking and streaming, the backend maintains conversation context, and the LLM performs the guessing-game dialogue. The overall responsibility structure will be:



i.

13. API

- a. The existing: POST /llmchat, will remain the primary API path for both games. The existing OpenAIRequest format will continue to be used. Normal initialization messages, user guesses, editable system prompts, and other game messages will all travel through /llmchat. No City-specific or Landmark-specific API will be introduced. Therefore, there will not be separate endpoints. Both games follow:
 - i. City
 - ii. → /llmchat
 - iii. Landmark
- b. Any backend-clearing mechanism used for clear() and switchGame() will remain separate from normal game messages and will clear the conversation associated with the application. The original design proposed /llmclear using the existing OpenAIRequest format; that design should be retained if /llmclear is the clearing mechanism allowed by the assignment/tutorial backend requirements.

Honor Pledge

Honor Code Pledge:

"I [HAVE] received aid from an LLM or automated coding assistant on this submitted work. If I received aid, the specific tool used was (specify model and version): [Chat GPT version 5.6_]. Regardless of assistance, I fully understand and can defend all code and design choices within this submission, and I have not concealed any violations of the Honor Code."

Signed: _____Lena Armijo_____ Date: _____9/29/26_____